```
Output:
Array
(
    [Alice] => 1
    [Bob] => 2
)
PHP Warning:  array_count_values(): Can only count string and integer
values, entry skipped...
```

`$arr` is a simple flat array. `'Alice'` occurs once, and `'Bob'` occurs twice. The output of `array_count_values()` is an associative array where each key is a distinct value of the input array, and each value is an integer count of the number of times that value is found. However, `$struct` – a multi-dimensional array – generates a "PHP Warning" message. `array_count_values()` only works if the value is an integer or a string (which makes sense because PHP arrays can only have integers or strings as keys).

What if we wanted to evaluate objects (that couldn't be coerced to string values), or floats? We've reached the limits of PHP's built-in array functions at this point. We need to write our own code and have it execute for all elements in the structure. There is a classic solution for a program that needs to traverse a multi-dimensional data structure, the recursive function call:

```
$struct = [
    ['Alice', 'Bob', 'Bob'],
    ['Alice', 'Bob', 'Bob'],
    ['Alice', 'Russell', 'Bob'],
];

print_r(recursive_array_count($struct));

function recursive_array_count(array $array, array &$ret = []): array {
    foreach ($array as $ele) {
        if (is_array($ele)) {
            recursive_array_count($ele, $ret);
        } else{
            (isset($ret[$ele])) ? $ret[$ele]++ : $ret[$ele] = 1;
        }
    }
```